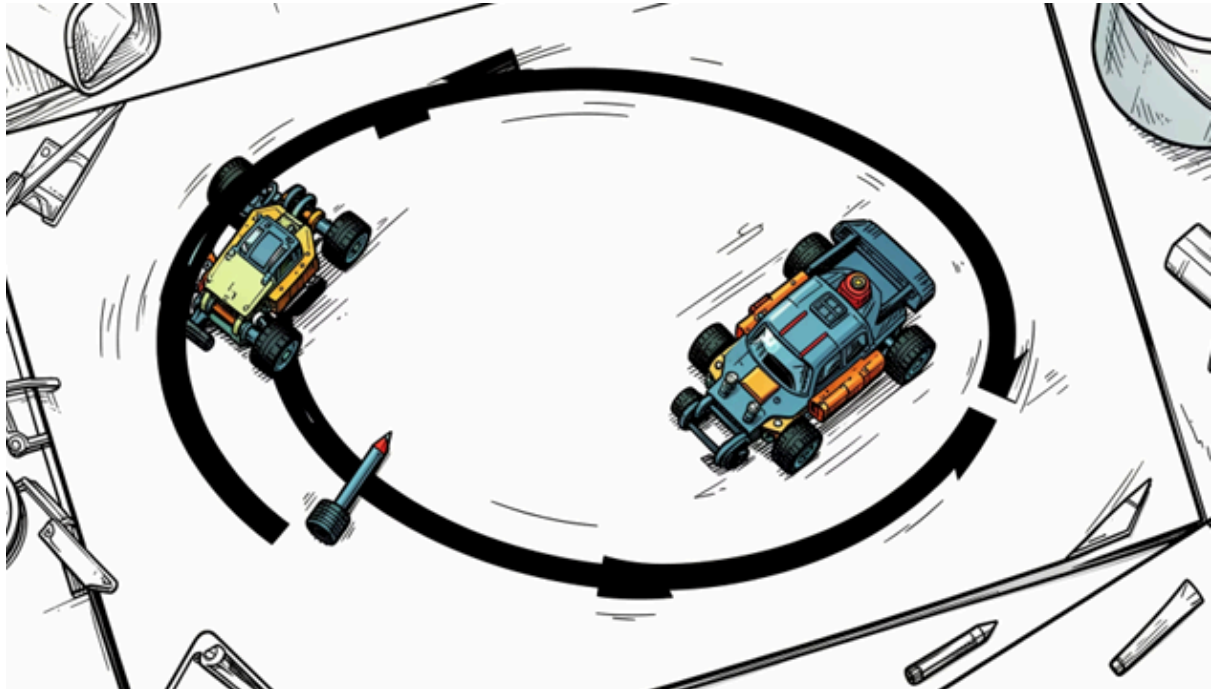


# VEHÍCULO AUTÓNOMO SEGUIDOR DE LÍNEA



**Integrantes: Javier Maya-Lucca Rando-Hernan Lombino**

## 1. Introducción

El presente informe documenta el diseño, desarrollo e implementación de un vehículo autónomo seguidor de línea controlado por el microcontrolador **ESP32**. El sistema es capaz de corregir su trayectoria en tiempo real para seguir un circuito de línea negra sobre un fondo blanco. Para la adquisición de datos del entorno se utilizaron dos sensores ópticos reflexivos **CNY70**, cuyos valores analógicos fueron relevados experimentalmente y procesados mediante los canales de conversión analógico-digital (ADC) del microcontrolador. La etapa de potencia y locomoción se basa en una configuración de tracción diferencial con dos motores de corriente continua (CC), controlados en sentido mediante lógica de puente H y en velocidad mediante modulación por ancho de pulsos (PWM).

## 2. Objetivos del Proyecto

- Diseñar e integrar un sistema mecatrónico basado en el microcontrolador ESP32.
- Realizar el relevamiento empírico de las curvas de respuesta analógica de los sensores ópticos frente a diferentes materiales y colores.
- Desarrollar un algoritmo de toma de decisiones en tiempo real basándose en un umbral crítico de calibración optimizado.
- Controlar la velocidad y el sentido de giro de motores de CC de forma independiente empleando señales digitales y PWM.
- 

## 3. Descripción del Hardware y Conexión Eléctrica

El sistema se compone de tres bloques integrados: censado (entradas), procesamiento (control) y actuación (potencia).

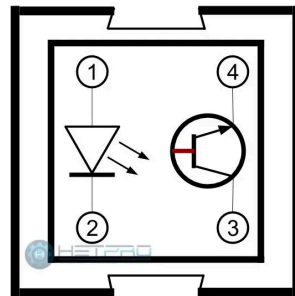
### 3.1. Unidad de Procesamiento (ESP32 DevKit)

Se seleccionó la plataforma ESP32 debido a sus altas prestaciones para control en tiempo real:

- **Procesador:** Dual-core a 240 MHz, ideal para el procesamiento simultáneo de periféricos.
- **Conversor Analógico Digital (ADC):** Configurado a una resolución de 12 bits, lo que proporciona un rango de lecturas discretas de 0 a 4095.
- **Acondicionamiento de Señal:** Se estableció por software una atenuación de **11 dB**, expandiendo el rango de lectura del ADC hasta un máximo cercano a los 3.3V de la lógica del chip.

### 3.2. Configuración Eléctrica de los Sensores CNY70

Cada sensor CNY70 acopla un diodo emisor infrarrojo y un fototransistor. De acuerdo al diseño eléctrico implementado en el laboratorio, los fototransistores se conectaron en configuración donde al diodo se le colocó una resistencia de protección de 330 ohm y por medio de un divisor resistivo utilizando resistencias fijas de 6.2K Ohm la salida del fototransistor al ADC del ESP32.



- Cuando el sensor se posiciona sobre una superficie clara, el fototransistor conduce intensamente hacia masa, haciendo que la tensión del nodo de lectura disminuya (valores analógicos altos reflejados en el ADC debido a la inversión o la reflectancia directa según el acoplamiento).
- Las salidas de señal de los divisores resistivos se derivaron directamente a los pines **GPIO 39 (Sensor Izquierdo)** y **GPIO 36 (Sensor Derecho)** del ESP32.

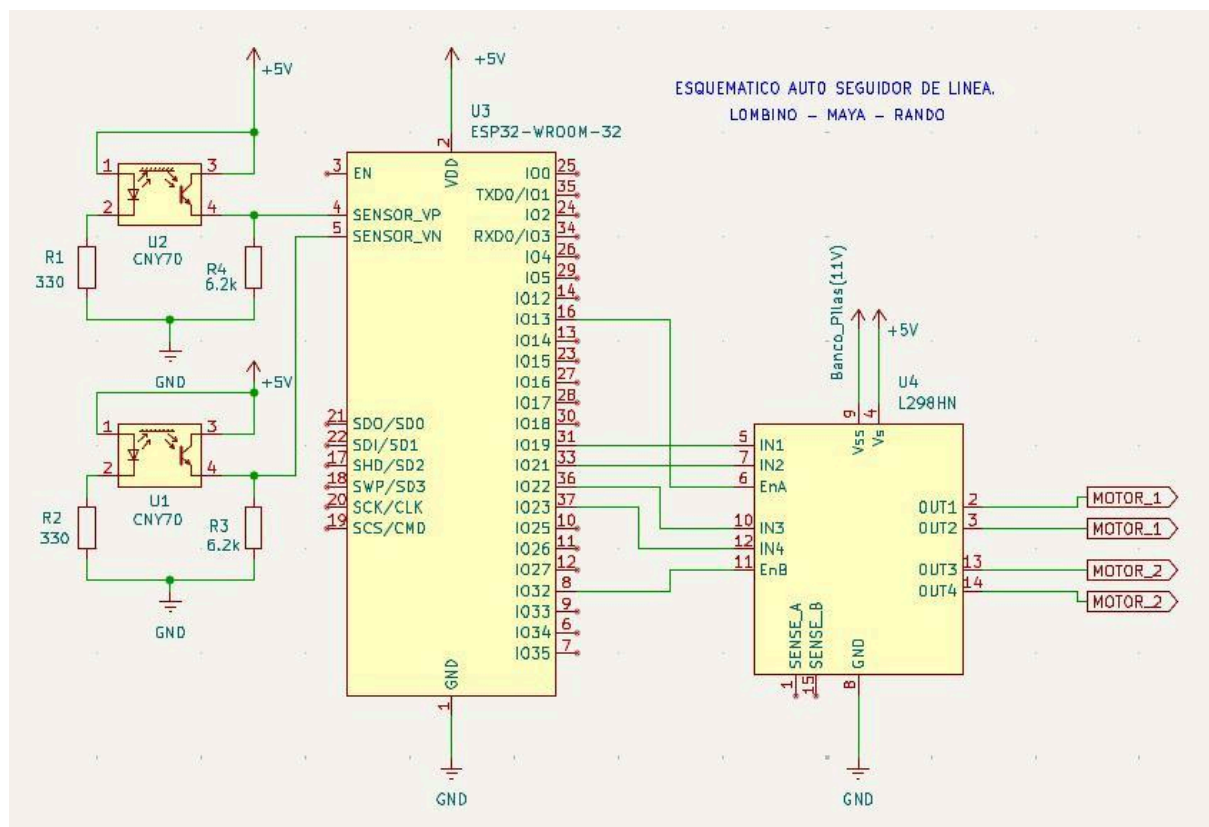
### 4. Asignación de Pines (Pinout)

Para la conexión física del prototipo se utilizó la siguiente distribución de pines del ESP32:

| Componente / Bloque     | Función de Control         | Pin GPIO ESP32 | Configuración en Código |
|-------------------------|----------------------------|----------------|-------------------------|
| <b>Sensor Izquierdo</b> | Entrada Analógica ADC (VN) | <b>GPIO 39</b> | INPUT (Por defecto ADC) |
| <b>Sensor Derecho</b>   | Entrada Analógica ADC (VP) | <b>GPIO 36</b> | INPUT (Por defecto ADC) |
| <b>Motor Derecho</b>    | Dirección 1 (IN1)          | <b>GPIO 21</b> | OUTPUT                  |
| <b>Motor Derecho</b>    | Dirección 2 (IN2)          | <b>GPIO 19</b> | OUTPUT                  |

|                        |                   |                |        |
|------------------------|-------------------|----------------|--------|
| <b>Motor Derecho</b>   | Velocidad (PWM)   | <b>GPIO 13</b> | OUTPUT |
| <b>Motor Izquierdo</b> | Dirección 1 (IN1) | <b>GPIO 23</b> | OUTPUT |
| <b>Motor Izquierdo</b> | Dirección 2 (IN2) | <b>GPIO 22</b> | OUTPUT |
| <b>Motor Izquierdo</b> | Velocidad (PWM)   | <b>GPIO 32</b> | OUTPUT |

#### 4.1 Circuito electrónico esquemático de interconexión.



#### 5. Ensayos de Laboratorio y Calibración (Datos Empíricos)

Con el fin de determinar de manera científica el valor óptimo para el umbral de discriminación del software, se realizaron pruebas dinámicas midiendo la respuesta cruda del ADC sobre diferentes materiales, texturas y colores. Los resultados obtenidos se detallan a continuación:

### 5.1. Tabla de Mediciones de ADC según Superficie

A continuación se vuelcan las lecturas obtenidas en los canales ADC del dispositivo bajo una resolución de 12 bits:

| Superficie /<br>Material de Prueba      | Lectura ADC 1<br>(Izquierdo) | Lectura ADC 2<br>(Derecho) | Condición del<br>Entorno   |
|-----------------------------------------|------------------------------|----------------------------|----------------------------|
| Negro sobre Cinta<br>Aisladora          | 150                          | 245                        | Absorción Máxima           |
| Negro Fibrón<br>sobre Papel<br>Blanco   | 160                          | 336                        | Absorción Nominal          |
| Negro Mesa /<br>Filmina<br>Transparente | 430                          | 790                        | Absorción con Brillo       |
| Gris Papel Oscuro                       | 820                          | 993                        | Zona de Transición         |
| Gris Papel Blanco                       | 1580                         | 2385                       | Reflectancia Media         |
| Verde Brillante                         | 2020                         | 3155                       | Reflectancia<br>Media-Alta |
| Blanco Papel                            | 2760                         | 3880                       | Reflectancia<br>Máxima     |

### 5.2. Justificación Técnica del Umbral Técnico (= 800)

El análisis de los datos experimentales justifica plenamente la selección del valor **800** como el umbral crítico en el código fuente:

1. **Margen de Seguridad de la Línea:** Todas las superficies negras probadas (cinta aisladora y fibrón) arrojan valores significativamente inferiores a 800 (entre 150 y

336). Incluso en la condición más desfavorable (mesa negra con filmína, que genera reflexiones parásitas), el sensor derecho se mantuvo en 790, justo por debajo del límite.

2. **Discriminación de Fondo:** Las superficies claras (papel blanco, fondos grises o de color verde brillante) superan ampliamente el valor de 800, llegando hasta un máximo de 3880.
3. **Análisis de Asimetría de los Componentes:** Se observa una marcada diferencia entre ambos sensores; el ADC 2 (Derecho) presenta de manera sistemática lecturas más elevadas que el ADC 1 (Izquierdo). Por ejemplo, sobre papel blanco el sensor derecho entrega un valor un 40% superior al izquierdo. Esta asimetría confirma que un umbral de 800 es un excelente "punto de compromiso" para asegurar que ambos canales conmutan correctamente de estado sin necesidad de sobrecargar el código con calibraciones individuales complejas.

## 6. Lógica de Decisiones Cinemáticas

El programa evalúa las lecturas en cada iteración del bucle principal y aplica la siguiente matriz de comportamiento para corregir el rumbo mediante tracción diferencial:

| Estado Izquierdo (Vizq>800) | Estado Derecho (Vder>800) | Interpretación del Entorno            | Acción del Vehículo    | Ciclo PWM (Izq / Der) |
|-----------------------------|---------------------------|---------------------------------------|------------------------|-----------------------|
| 0 (Negro)                   | 0 (Negro)                 | Ambos sensores centrados en la línea. | Avanzar en línea recta | 80 / 80               |
| 0 (Negro)                   | 1 (Blanco)                | El vehículo se desvía a la derecha.   | Girar a la Izquierda   | 110 / 110             |
| 1 (Blanco)                  | 0 (Negro)                 | El vehículo se desvía a la izquierda. | Girar a la Derecha     | 110 / 110             |
| 1 (Blanco)                  | 1 (Blanco)                | Pérdida de pista o fin de circuito.   | Detención Inmediata    | 0 / 0                 |

*Nota de diseño:* La velocidad asignada para las maniobras de giro (110) es configurada mayor que la velocidad de avance nominal (80) para proveer el torque necesario para vencer la inercia rotacional y el rozamiento dinámico en curvas cerradas.

## 7. Código Fuente Comentado

A continuación se expone el código de producción desarrollado para el sistema, diseñado bajo el paradigma de máquina de estados por umbralización:

```
//int slderecho = 18; //declaro el pin de entrada del sensor derecho
//int slizquierdo = 17; //declaro el pin de entrada del sensor izquierdo
//int Vslderecho = 0; //declaro valor inicial del sensor derecho
//int Vslizquierdo = 0; //declaro valor inicial del sensor izquierdo
int Dmotorderecho1 = 21; // declaro el pin para manejar giro de motor derecho
int Dmotorderecho2 = 19; // declaro el pin para manejar giro de motor derecho
int PWMmotorderecho = 13; // declaro el pin para manejar velocidad del motor derecho
int Dmotorizquierdo1 = 23; // declaro el pin para manejar giro de motor izquierdo
int Dmotorizquierdo2 = 22; // declaro el pin para manejar giro de motor izquierdo
int PWMmotorizquierdo = 32; // declaro el pin para manejar velocidad del motor izquierdo
int ADCderecho = 36; // declaro pin ADC para sensor derecho
int ADCizquierdo = 39; // declaro pin ADC para sensor izquierdo
int VADCderecho = 0; // declaro valor inicial de ADC derecho
int VADCizquierdo = 0; // declaro valor inicial de ADC izquierdo
int umbral = 800; // negro < 800 , blanco > 800
int Vfinalderecho = 1;
int Vfinalizquierdo = 1;

void setup()
{
    Serial.begin(9600);

    analogReadResolution(12); // Configuramos la resolución a 12 bits (rango de 0 a 4095)

    analogSetAttenuation(ADC_11db); // Rango hasta ~3.3V

    //pinMode (slderecho, INPUT); // declaro pin 14 como entrada
    //pinMode (slizquierdo, INPUT); // declaro pin 27 como entrada

    pinMode (Dmotorderecho1, OUTPUT); // declaro pin como salida
    pinMode (Dmotorderecho2, OUTPUT); // declaro pin como salida
    pinMode (PWMmotorderecho, OUTPUT); // declaro pin como salida
    pinMode (Dmotorizquierdo1, OUTPUT); // declaro pin como salida
```

```

pinMode (Dmotorizquierdo2, OUTPUT);// declaro pin como salida
pinMode (PWMMotorizquierdo, OUTPUT);// declaro pin como salida

}

void loop() {
  //Vslderecho=digitalRead(slderecho); // leo el valor digital de pin 14
  //Serial.print("El valor del sensor derecho es:"); // escribo el valor en consola
  //Serial.println(Vslderecho);
  //Vslizquierdo=digitalRead(slizquierdo); // leo el valor digital de pin 27
  //Serial.print("El valor del sensor izquierdo es:"); // escribo el valor en consola
  //Serial.println(Vslizquierdo);

  VADCderecho = analogRead(ADCderecho); // Lectura del ADC del ESP32 (Resolución de 12
bits: 0 a 4095)
  VADCizquierdo = analogRead(ADCizquierdo); // Lectura del ADC del ESP32 (Resolución de
12 bits: 0 a 4095)

  // Determinar estado lógico (0 = Blanco, 1 = Negro)
  // Si el valor supera el umbral, está sobre el fondo blanco.
  if (VADCderecho > umbral)
  {
    Vfinalderecho = 1;
  }
  else
  {
    Vfinalderecho = 0;
  }
  if (VADCizquierdo > umbral)
  {
    Vfinalizquierdo = 1;
  }
  else
  {
    Vfinalizquierdo = 0;
  }
  if(Vfinalizquierdo == 1 && Vfinalderecho == 1 ) // si los dos sensores están sobre blanco SE
DETIENE
  {
    digitalWrite(Dmotorderecho1, LOW);
    digitalWrite(Dmotorderecho2, LOW);

    digitalWrite(Dmotorizquierdo1, LOW);
    digitalWrite(Dmotorizquierdo2, LOW);
    //analogWrite(PWMMotorderecho, 40); // velocidad del motor derecho con PWM

```



```

    //analogWrite(PWMmotorizquierdo, 40); // velocidad del motor izquierdo con PWM
}
if (Vfinalizquierdo ==1 && Vfinalderecho ==0 ) // si izquierdo esta blanco y el derecho en
negro gira a la DERECHA
{
    digitalWrite(Dmotorderecho1, HIGH);
    digitalWrite(Dmotorderecho2, LOW);

    digitalWrite(Dmotorizquierdo1, LOW);
    digitalWrite(Dmotorizquierdo2, HIGH);

    analogWrite(PWMmotorderecho, 110); // velocidad del motor derecho con PWM
    analogWrite(PWMmotorizquierdo, 110); // velocidad del motor izquierdo con PWM
}

if (Vfinalizquierdo==0 && Vfinalderecho==1 ) // si izquierdo esta negro y el derecho en
blanco gira a la IZQUIERDA
{
    digitalWrite(Dmotorderecho1, LOW);
    digitalWrite(Dmotorderecho2, HIGH);

    digitalWrite(Dmotorizquierdo1, HIGH);
    digitalWrite(Dmotorizquierdo2, LOW);

    analogWrite(PWMmotorderecho, 110); // velocidad del motor derecho con PWM
    analogWrite(PWMmotorizquierdo, 110); // velocidad del motor izquierdo con PWM
}

if (Vfinalizquierdo==0 && Vfinalderecho==0 ) // si izquierdo esta NEGRO y el derecho en
NEGRO sigue recto
{
    digitalWrite(Dmotorderecho1, LOW);
    digitalWrite(Dmotorderecho2, HIGH);

    digitalWrite(Dmotorizquierdo1, LOW);
    digitalWrite(Dmotorizquierdo2, HIGH);
    analogWrite(PWMmotorderecho, 80); // velocidad del motor derecho con PWM
    analogWrite(PWMmotorizquierdo, 80); // velocidad del motor izquierdo con PWM

}}

```

## 8. Conclusiones

1. **Validación de Hipótesis:** Las pruebas empíricas de laboratorio confirmaron la efectividad del método analógico sobre el digital para entornos con variaciones de

reflectancia. El umbral de 800 ofrece un comportamiento binario robusto frente a superficies altamente disímiles.

2. **Compensación de Hardware:** El análisis de asimetría detectado en los sensores demostró la importancia de realizar relevamientos en hardware antes de fijar parámetros estrictos de software.